商品搜索：从 LIKE 到 ES + Milvus hybrid

用代码 + 业务双视角讲清 gomall 的搜索全链路

gomall · deck 03 (v2 expanded)

今日大纲

搜索是流量入口，不是普通功能

ES 集群、索引设计与排序

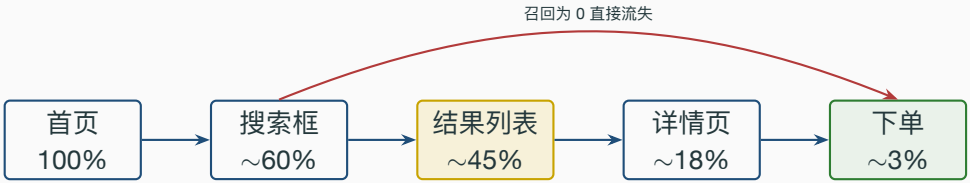
上架到可搜：outbox + 异步索引 + 兜底

语义层：Milvus + Embedding + 故障预案

业务侧总结

搜索是流量入口，不是普通功能

用户进站后第一件事就是搜



- 进站用户里约 60% 会主动用搜索，是漏斗最宽的一跳。
- ” 搜不到 / 搜偏” 用户不会换词，直接退出 App。
- 入口处的相关性，决定后面三跳还能不能继续。

routes/router.go:41 /product/search 是入口；:42 /product/semantic-search 是 hybrid 入口

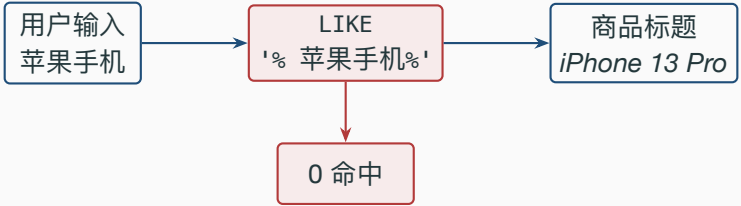
业务对搜索的四条硬约定

- **召回**：用户搜得到的词，能跑出至少一条相关结果。
- **时延**：p95 < 200ms，列表页才不会”卡半秒空白”。
- **新鲜度**：商家上架 → 站内可搜，SLA ≤ 5min。
- **兜底**：ES 挂掉，搜索接口不能 5xx，必须有降级路径。

> 这四条决定了搜索系统不能只是 `SELECT * WHERE name LIKE '%kw%'`。

stressTest/REPORT.md：/product/show 同库 PK 查询基线 62,226 RPS / p95 3ms 作为时延锚点

用户搜” 苹果手机”，DB 搜不到 iPhone 13



- 用户脑里是中文，商家上架习惯写英文型号。
- LIKE 不知道” 苹果 == Apple == iPhone”，子串不匹配 = 0 命中。
- 这不是用户拼错，是搜索系统欠用户一个” 分词 + 同义” 能力。

repository/db/dao/product.go:80–98 是当前 DB 兜底用的 LIKE 实现

LIKE 的第二个问题：跑不动

指标	数值
product 表行数	956K 行
product 表磁盘大小	364 MB
LIKE `'%X%'` 能走索引?	否（前缀通配）
全表扫预估单 query 耗时	秒级

- `LIKE '%kw%'` 前缀是通配符，B+ 树用不上，必扫全表。
- 96 万行 + 364 MB 数据，单查就吃满一颗 CPU。
- 对照：同库 `/product/show` 走主键命中 **62,226 RPS / p95 3ms**。

stressTest/REPORT.md § 数据规模 + 链路压测表第 1 行

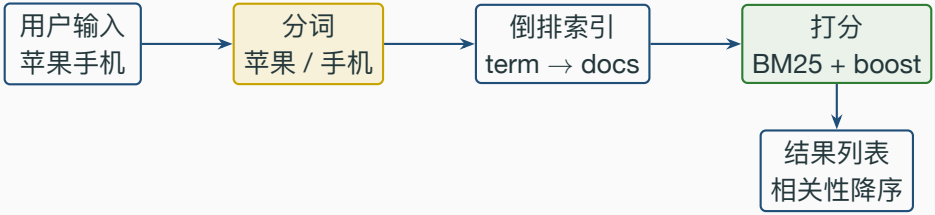
LIKE 缺的三件事，全是业务能力

能力	LIKE 有?	业务后果
分词	无	” 苹果手机” 搜不到”iPhone”
字段加权	无	标题命中 vs 详情命中等权排
高亮	无	列表里看不到为什么命中

- 分词决定召回，字段加权决定排序，高亮决定点击率。
- 三件事任一缺失，” 搜索” 就退化成” 碰运气包含”。
- 这才是 gomall 换 ES 的业务动机，不是图组件新潮。

ES 集群、索引设计与排序

ES 替 LIKE 做什么



- **分词**：按词切（中文走 IK 或 standard fallback）。
- **倒排索引**：term 直接索引到 doc 列表，避开全表扫。
- **打分**：`multi_match` + 字段加权，name > title > info。

repository/es/product_index.go:163–227 SearchProductsWithScore 多字段加权查询

字段加权背后的业务判断

字段	业务含义	加权
`name`	商品主名（用户预期匹配）	$\times 3$
`title`	商品标题（含型号 / 规格）	$\times 2$
`info`	商品详情（长文，噪声多）	$\times 1$

- 用户搜”iPhone”，” 标题就叫 iPhone 13” 应当排在”info 提到 iPhone” 前面。
- 加权由运营调，不是开发拍。大促可临时把 `title` 上调到 $\times 4$。
- 不加权 $\rightarrow$ ” 标签店家把热词塞进 info” 会霸占第一屏。

repository/es/product_index.go:174–177 `fields={name^3,title^2,info}`

代码段 1：ES 关键词检索（含 _score 输出）

```
170 must := []map[string]any{{
171    "multi_match": map[string]any{
172       "query": keyword,
173       "fields": []string{"name^3", "title^2", "info"},
174    },
175 }}
176 filter := []map[string]any{}
177 if categoryID != nil {
178    filter = append(filter, map[string]any{
179       "term": map[string]any{"category_id": *categoryID},
180    })
181 }
182 q := map[string]any{
183    "from": from, "size": size,
184    "query": map[string]any{"bool": map[string]any{
185       "must": must, "filter": filter,
186    }},
187 }
```

repository/es/product_index.go:170–191 关键片段

代码段 1 逐行讲解

- **L170–175**：`multi_match` 同时查 name / title / info，靠 `^N` 给字段加权。这一行就是上一页那张加权表的代码落点。
- **L176–180**：`filter` 走 `term`，类目过滤不参与打分，只做 yes/no 过滤，性能更稳。运营按类目限定时进这条分支。
- **L182–189**：`bool.must + filter` 是 ES 的标准组合——must 决定相关性，filter 决定可见性。两者分离，分页和打分才不会互相干扰。
- **业务对应**：name 权重最高，对应”用户最在意主名命中”；filter 不打分，对应”类目是硬过滤，不是建议”。

must / should / filter / must_not 四件套

子句	是否打分	典型用途
`must`	是	关键词召回，决定相关性
`should`	是	可选加分项，命中即上调
`filter`	否	硬过滤：类目 / 上架 / 库存 >0
`must_not`	否	黑名单：下架 / 违禁 / 自家店

- 打分子句进 query cache miss 时重算，filter 子句进 filter cache 长期命中。
- 把所有”硬条件”塞进 filter 是最常见的性能优化，可使 QPS 翻倍。
- `should` 配合 `minimum_should_match` 还能做”至少命中 N 个同义词”。

ES 官方 query DSL 与 gomall 用法已对齐：repository/es/product_index.go:170–191

BM25 vs TF-IDF：现代 ES 默认值

- TF-IDF：`tf` 直接线性，长文档天然占便宜。
- BM25：`tf` 走饱和函数 `(k+1)*tf / (k+tf*…)`，词频再高也不会无脑加分。
- BM25 引入文档长度归一 `b`，长 info 不再压制短 name。
- ES 7.x 起默认 `similarity=BM25`，gomall 不显式覆盖，保留默认。

> BM25 是”被工业界打磨过 20 年的 TF-IDF”；运营提”搜索结果偏长描述”时，先验证是不是误改了 `k1/b`。

repository/es/product_index.go:20–41 mapping 未覆盖 similarity，沿用集群默认 BM25

function_score 与 boost：业务分挂在相关性上

机制	典型用途
`function_score`	相关性 × 销量 / 评分 / 时效
`fields:"name^3"`	字段级权重（gomall 当前用法）
顶层 `boost`	A/B 桶整路召回乘系数
`negative_boost`	命中黑词不杀只压

- function_score 把业务分挂在 BM25 外面，不污染相关性调试。
- 字段级 boost 最稳，运营改的就是 `name^3`；大促把 title 调到 `^4`。
- 改过的 boost 必须打到 dashboard，否则后人查” 为什么排序变了” 会无从下手。

当前 *fields={name^3,title^2,info}*；*function_score* 接入点 *service/search/semantic.go:107–111*

把” 召回 vs 排序” 拆开

- 召回管” 宁可错杀不可漏掉”，几百到几千条。
- 粗排在 ES 里就完成，毫秒级。
- 精排是 PS / 用户特征上的二次打分，几十毫秒，命中率才是 KPI。

gomall 当前实现到粗排为止；精排留作后续 *PR*，接入点见 *service/search/semantic.go:107–111*

个性化排序：三类特征 + 挂载点

特征类	例子	来源
用户画像	性别 / 年龄段 / 价格带偏好	user profile 表
历史点击	7 天内点过的类目 / 品牌	点击日志 + 实时流
协同信号	同类用户买过什么	离线 i2i / u2i 矩阵

- 历史点击走”近因衰减” `exp(-(now-ts)/τ)`，τ 取 3–7 天。
- 挂载点：`h.Score += wPersonal * personalScore(uid, h.Product)`，加在融合分外面。
- Redis miss 走默认 0（保中性），永远比”用户画像污染基线”安全。

service/search/semantic.go:107–111 融合分赋值处是个性化加权的唯一注入点

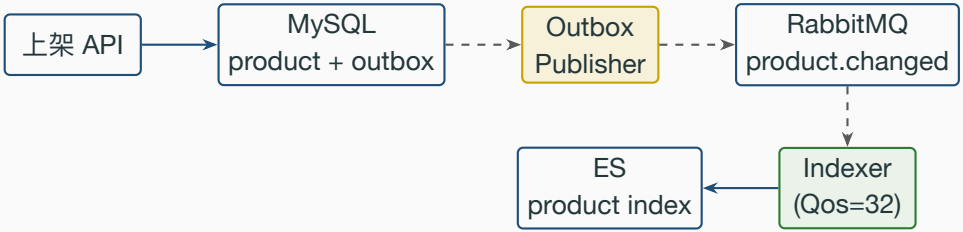
上架到可搜：outbox + 异步索引 + 兜底

业务承诺：商家上架 5 分钟内必须搜得到

- 商家后台点”上架” → 站内搜索能搜到，业务侧的 SLA 是 $\leq$ 5min。
- 5min 是商家容忍上限：拍照 / 调价 / 改文案的反馈节奏。
- 写 DB 同步写 ES 行不行？**不行**：ES 抖动直接 fail 上架，商家退出。
- 必须**解耦**：DB 写完就返回成功，ES 异步追上。

service/product.go:155, 253, 289 三处商品写入后调 emitProductChanged

outbox → MQ → indexer 的链路



- 上架事务里只写 product + outbox，二者同库同事务，原子。
- Publisher 后台扫 outbox → 投递到 `product.changed` 路由。
- Indexer 消费后写 ES。Qos=32 控制并发，避免 ES 被写穿。

service/product.go:257–265 emitProductChanged；service/search/indexer.go:14–49 indexer 主循环

代码段 2：indexer 主循环

```go
17 func StartProductIndexer(ctx context.Context) error {
18    rabbitmq.BindDomainQueue(indexerQueue, "product.changed")
19    ch, _ := rabbitmq.GlobalRabbitMQ.Channel()
20    ch.Qos(32, 0, false) // 在途上限 32 条
21    msgs, _ := ch.Consume(indexerQueue, "", false, false, false, false, nil)
22    go func() {
23       for d := range msgs {
24          var ev events.ProductChanged
25          if json.Unmarshal(d.Body, &ev) != nil {
26             d.Nack(false, false); continue // 脏消息进死信
27          }
28          if handleProductChanged(ctx, ev) != nil {
29             d.Nack(false, true); continue // 重投自愈
30          }
31          d.Ack(false)
32       }
33    }()
34    return nil
35 }
```

service/search/indexer.go:17–49（折行省略错误处理）

代码段 2 逐行讲解

- **L18–19**：`BindDomainQueue` 把 `product.changed` 路由绑到 `search.product.indexer` 队列。上架 / 改价 / 下架共用一条路由。
- **L22**：`Qos(32, 0, false)` 让消费端最多在途 32 条未 ack。这是给 ES 写入设的”流量阀”，对应 deck 05 的”在网关之外，每个消费者也要节流”。
- **L30–34**：JSON 解析失败 → `Nack(false, false)`，不重投。脏消息进死信，不阻塞健康消息。
- **L35–38**：业务处理失败 → `Nack(false, true)` 重投，给 ES 短抖动一次自愈机会。
- **L39**：成功才 ack，保证 at-least-once。索引是幂等的（同 docID 覆盖），重复消费无害。

service/search/indexer.go:25 Qos=32；:36/:42 Nack 语义

outbox → ES 端到端延迟拆解 + 幂等

环节	典型值
上架事务 commit	5–20 ms
Outbox publisher 轮询周期	1 s
RabbitMQ 投递 + 队列等待	10–500 ms
Indexer 写 ES `UpsertProduct`	20–50 ms
ES `refresh_interval` 可见窗口	$\leq$ 1 s
合计 (p95)	**~2 s**
合计 (p99, RMQ 堆积)	**~30 s**

- 5min SLA 还有 10× 余量，吸收 RMQ 堆积告警前的窗口。
- `DocumentID = product.ID` + `Refresh:false` 同 ID 覆盖 + 1s 刷新窗口，幂等 + 吞吐两兼顾。

service/search/indexer.go:51–60 handleProductChanged；repository/es/product_index.go:105–131

代码段 3：product mapping（建索引时的 schema）

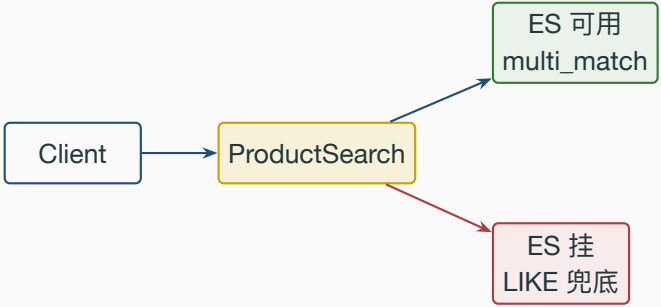
```
20 "settings": { "number_of_shards": 1, "number_of_replicas": 0 },
21 "mappings": { "properties": {
22   "name": { "type": "text", "analyzer": "standard" },
23   "title": { "type": "text", "analyzer": "standard" },
24   "info": { "type": "text", "analyzer": "standard" },
25   "category_id": { "type": "long" },
26   "price": { "type": "keyword" },
27   "on_sale": { "type": "boolean" },
28   "created_at": { "type": "date", "format": "epoch_second" }
29 }}
```

repository/es/product_index.go:20–41 productIndexMapping（节选）

代码段 3 逐行讲解

- **shards=1 / replicas=0**：gomall 单节点部署，不做多副本。生产换多节点要把 replicas 拉到 1，read 走主 + 副两副本。
- **name/title/info 是 `text`**：参与分词 + 倒排，是关键词召回的目标字段。
- **price 是 `keyword`**：不分词，整串做 filter / exact match。价格永远不会被” 分词” 成”99” + ” 元”。
- **category_id / boss_id 是 `long`**：做 `term` 过滤（按类目 / 按店铺筛），不打分。
- **on_sale 是 `boolean`**：下架商品要从结果里隐藏，靠这个字段做硬过滤。
- **analyzer=standard**：未装 IK 分词器时的 fallback，保证集群裸装也能跑通。

业务约定：搜索接口不许 5xx，启动也不许阻塞



- 5xx 等于 app 崩页，比” 结果不准” 业务损失大得多。LIKE 慢也能用。
- `tryInitES` 用 `defer recover()` 把启动 panic 压平成 warn ——ES 故障不阻塞启动。
- **ES 是优化项，不是依赖项。**

service/product.go:294–323 兜底；cmd/main.go:64–73 tryInitES；initialize/search.go:13–31

语义层：Milvus + Embedding + 故障预案

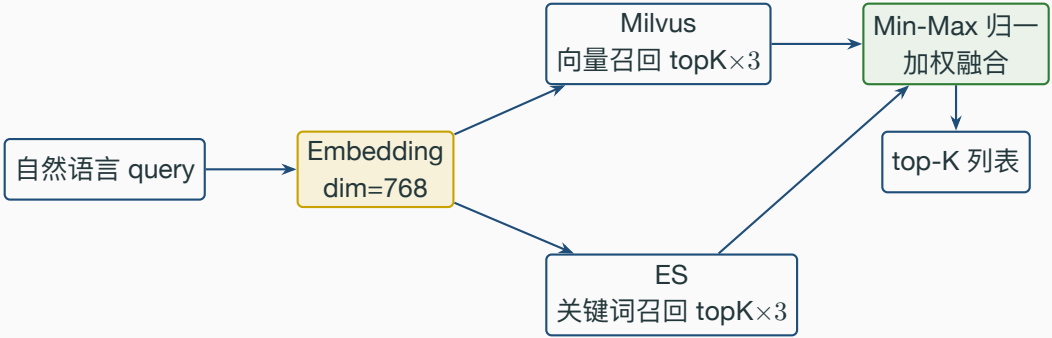
ES 关键词搞不定的三类 query

query 类型	例子	ES 行为
自然语言	送女朋友的礼物	拆成” 送/女朋友/礼物” → 命中乱
跨表达同义	苹果手机最新款	名义命中 iPhone 15 漏
口语描述	适合健身的运动鞋	长尾词命中率趋近 0

- ES 衡量**词重合度**，不是**语义接近度**。
- 用户被短视频 / LLM 养成自然语言搜的习惯，长尾流量比例上升。
- 长尾召回率掉到 60% 以下，整段 GMV 静默漏走，没有告警。

service/search/semantic.go:1–56 hybrid 入口注释与默认权重

hybrid 三层：关键词 + 向量 + 业务调权



向量管召回，关键词管精度，融合权重管业务偏好。*service/search/semantic.go:54–143*

semanticSearchWith 主流程

融合权重：业务调权的入口

权重 (sem : kw)	行为	适合的 query
80 : 20	偏语义	” 送女朋友的礼物” 自然语言
50 : 50	中庸	gomall 默认
20 : 80	偏关键词	”iPhone 13 256G” 品牌型号
0 : 100	退化为 ES	ES 全场景兜底

- 50/50 是无偏起点，留 A/B 空间；短 query 偏关键词，长 query 偏语义。
- 大促” 今日特卖” 强插不改算法：**在融合分外面加业务分**

```
Score = wSem*sem + wKw*kw + promote(id)
```

- 不污染长期相关性，运营随时下线。

service/search/semantic.go:21–23 权重常量；*:107–111 Score* 处可挂业务 *boost*

代码段 4：min-max 归一 + 加权融合

```go
89  semNorm := minMaxNormalize(vecScores(vecHits))
90  kwNorm := minMaxNormalize(esScores(keywordHits))
91  fused := make(map[uint]*types.ProductSemanticHit)
92  for i, h := range vecHits {
93     id := uint(h.ID); if id == 0 { continue }
94     fused[id].SemanticScore = semNorm[i]
95  }
96  for i, h := range keywordHits {
97     fused[h.Doc.ID].KeywordScore = kwNorm[i]
98  }
99  for id, h := range fused {
100    h.Score = weightSemantic*h.SemanticScore +
101            weightKeyword *h.KeywordScore
102 }
```

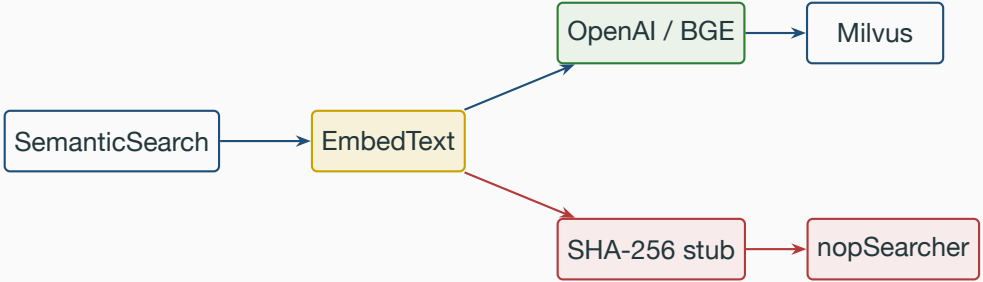
service/search/semantic.go:89–111 融合主体（折行示意）

代码段 4 逐行讲解

- **L89–90**：两路分数先各自 min-max 归一到 $[0, 1]$，避免 BM25 的 0–100 量纲压死向量的 0–1 量纲。
- **L93–99**：两路按 `product_id` 取并集——仅在一路出现的 ID 也计入，另一路缺失视为 0。
- **L100–103**：加权融合。weightSemantic / weightKeyword 是常量，A/B 时换 hash 分桶决定。
- **业务对应**：min-max 在小 batch（30–100 条）下稳定，单 query 内部即可计算，无需全局统计量。
- 全等分（如 ES miss 所有向量召回 fallback）时归一化结果给常量 1，意味着”两路一样可信，靠另一路决定排序”。

service/search/semantic.go:170–198 minMaxNormalize 全等集合返回 *1*

Embedding 与 Milvus 都做了可降级



- `EMBEDDING_API_URL` 未配 → SHA-256 stub 兜底；上线节奏 OpenAI → BGE 省 token；dim=768 硬约束。
- Milvus 走 `MilvusSearcher` 接口，未注入 → nop 返回 nil → hybrid 退化为纯 ES。
- **Milvus 挂只丢语义，关键词召回不受影响。**

service/search/embedding.go:52–113；service/search/milvus_stub.go:9–47；cmd/main.go:86–97 tryInitMilvus

HNSW 的三个核心参数

参数	含义	gomall 当前值
`M`	节点最大邻居数	16
`efConstruction`	建图时候选池大小	200
`efSearch`	查询时候选池大小	64

- `M` 大 → 图更密集，召回更准，但内存涨。常见 16–48。
- `efConstruction` 大 → 建图更慢但更优，回填速度敏感场景调低。
- `efSearch` 大 → 召回更准延迟更高，是**查询时**唯一可调旋钮。

repository/milvus/product_vector.go:25–28 hnswM=16 / hnswEfConstruction=200 / hnswEfSearch=64

代码段 5：HNSW 索引建立 + 查询参数

```go
// 建索引：M=16, efConstruction=200
idx, _ := entity.NewIndexHNSW(entity.L2, hnswM, hnswEfConstruction)
existing, _ := MilvusClient.DescribeIndex(ctx, ProductVectorCollection,
    productVectorVectorField)
if len(existing) == 0 {
    MilvusClient.CreateIndex(ctx, ProductVectorCollection,
        productVectorVectorField, idx, false)
}
MilvusClient.LoadCollection(ctx, ProductVectorCollection, false)

// 查询：efSearch=64
sp, _ := entity.NewIndexHNSWSearchParam(hnswEfSearch)
results, _ := MilvusClient.Search(ctx, ProductVectorCollection,
    []string{}, expr, []string{"id"},
    []entity.Vector{entity.FloatVector(queryVec)},
    "vector", entity.L2, topK, sp)
```

repository/milvus/product_vector.go:80–96 EnsureProductVectorCollection；:141–162 SearchProductVector

代码段 5 逐行讲解

- **L81**：`NewIndexHNSW(L2, M, efConstruction)` 决定索引结构；L2 是欧氏距离，与 BGE / text-embedding-3 默认归一化向量兼容。
- **L82–87**：先 `DescribeIndex` 判断有没有索引，没有才建。EnsureProductVectorCollection 是幂等的，重启不会重建。
- **L88**：`LoadCollection` 让 collection 进入内存。HNSW 索引是 in-memory 结构，没 load 就不能查。
- **L91–96**：`NewIndexHNSWSearchParam(efSearch)` 是**查询时**调旋钮，不需要重建索引就可以调召回精度/延迟权衡。
- **业务对应**：efSearch 是 SRE 故障时的”减压阀”，告警来了先调小，召回准确性损 1%–3% 换 P95 减半。

repository/milvus/product_vector.go:25–28 三个参数常量

HNSW vs IVF vs Annoy 选型

索引	召回率	查询延迟	内存开销
HNSW	95%–99%	1–5 ms (1M)	高（图结构）
IVF_FLAT	90%–95%	5–20 ms (1M)	中
IVF_PQ	80%–90%	2–10 ms (1M)	低（量化压缩）
Annoy	88%–94%	5–15 ms (1M)	中

- **HNSW**：默认选择，召回质量稳定，gomall 用这个。
- **IVF_FLAT/PQ**：10M+ 向量考虑，PQ 量化省内存但精度损失。
- **Annoy**：Spotify 系，磁盘友好，更新成本高。
- 选 HNSW 是工程默认，等到向量量级 >10M / 内存紧张再考虑 IVF_PQ。

repository/milvus/product_vector.go:80 NewIndexHNSW 是当前生效的索引类型

Embedding 模型选型矩阵

模型	维度	成本 (1M tokens)	中文表现
OpenAI text-embedding-3-small	1536→768	$0.02	中等
OpenAI text-embedding-3-large	3072→768	$0.13	优秀
BGE-base-zh-v1.5	768	自托管 GPU 成本	优秀
E5-large-v2	1024	自托管 GPU 成本	良好
SHA-256 stub (gomall 默认)	768	0	仅占位

- `embeddingDim=768` 是 mapping 硬约束，切模型必须降维到 768 才能复用 collection。
- 上线节奏：本地 stub → OpenAI 3-small（贴近 BGE）→ 自托管 BGE 省 token。
- BGE / E5 自托管的成本拐点：单日 query 量 >50 万次，自托管划算。

service/search/embedding.go:17–24 embeddingDim=768 / 默认 model=text-embedding-3-small

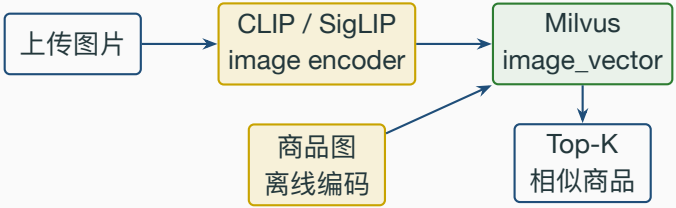
Embedding 切换的工程红线

- **维度不能变**：768 是 Milvus collection schema 硬约束，切模型先 reindex。
- **距离度量要对齐**：BGE 推 cosine，gomall 当前 L2；归一化向量后 L2 = 2 - 2cos，等价。
- **stub 必须保留**：CI / 本地起服务不能因为忘配 env 就 5xx。
- **超时 5s 硬上限**：`embeddingTimeout = 5 * time.Second`，OpenAI 卡顿不能拖垮搜索 P99。

> embedding 是**库外依赖**，切换前先在 staging 跑一遍 NDCG / MRR 评估，不要直接上生产。

service/search/embedding.go:18 embeddingDim=768；:19 embeddingTimeout=5s

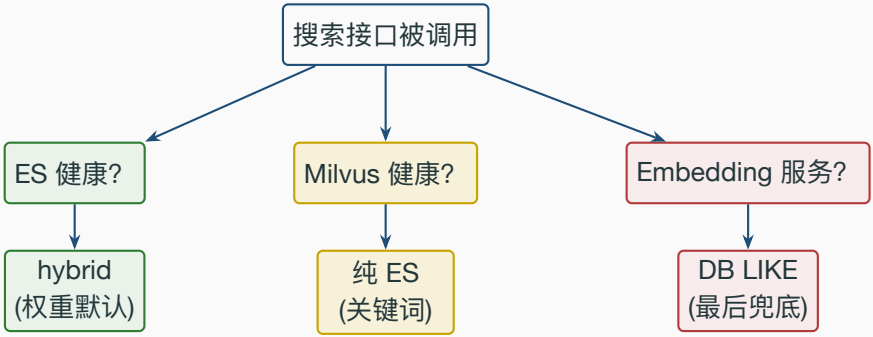
以图搜图路线图 (CLIP / SigLIP)



- CLIP / SigLIP 把图片和文本映射到同一向量空间，可做” 图找图”+” 文找图”。
- 落地：另建 `image_vector` collection（dim=512/768），与文本 collection 解耦。
- 商品图离线批量编码，写入走相同 outbox 路由，复用 indexer 模式。

规划落地点：*repository/milvus/* 复用 *product_vector* 模板新增 *image_vector*

三档降级决策树



ES + Milvus 都健康 → hybrid；Milvus 挂 → 纯关键词；ES 挂 → DB LIKE 兜底。

service/search/semantic.go:84–87 ES 失败时 keywordHits=nil；service/product.go:294–323 ES 全挂回 LIKE

容量规划：1M / 10M / 100M SKU

量级	ES 节点	Milvus 内存	索引重建时长
1M	1 node (4C8G)	4 GB (HNSW)	30 min
10M	3 nodes (8C16G)	32 GB (HNSW)	4 h
100M	5+ nodes (16C32G)	256 GB / 切 IVF_PQ	24 h+

- gomall 当前 **956K** SKU，落在 1M 档，单节点足够。
- 10M 是分水岭：ES 必须分片 + 副本；HNSW 内存接近 32 GB 拐点。
- 100M 切 IVF_PQ 量化索引，召回率换内存；同时考虑冷热分层。

stressTest/REPORT.md：product 956K 行 / 364MB 是 1M 档实测数据

业务侧总结

一张表把这套搜索系统讲完

能力	实现	代码位置
关键词召回	ES `multi_match`	repository/es/product_index.go
增量索引	outbox → MQ → indexer	service/search/indexer.go
DB 兜底	ES 客户端为 nil → LIKE	service/product.go:294
全量回填	admin 触发的 Backfill	service/search/backfill.go
语义召回	Milvus + embedding	service/search/semantic.go
模型切换	env: `EMBEDDING_API_URL`	service/search/embedding.go
启动静默	defer recover	cmd/main.go:64–73

routes/router.go:41–42 search + semantic-search 两个入口；:163 admin/search/backfill

四方视角 + 压测数据复核

- **用户 / 商家 / 运营 / SRE**：搜得到、5min 可搜、能调权、挂了不出 5xx。
- 同库 PK 查询能跑 **62,226 RPS / p95 3ms** (product/show)，证明慢的是**查询类型**（LIKE / count），不是数据库本身。
- 把” 模糊 + 排序” 搬到 ES，是把一个本不该 DB 干的活儿挪到合适的引擎。

副本系统判据：**副本旧一点点不要命，副本错乱才要命**。outbox 把” 上架成功” 和” 可搜” 解耦，是 5min SLA 的工程根据。

stressTest/REPORT.md：product 956K 行 / 364MB；/product/show 62,226 RPS / 3ms；/product/list 24.5 RPS / 2.50s

面试 Q&A (1/3)

Q1：为什么不让上架 API 直接写 ES，而要 outbox?

A：DB 与 ES 不同事务，同步写要么失败回滚（伤体验），要么不一致（伤召回）。outbox 把”DB 写成功” 和”ES 写成功” 两件事拆开，分别保证。代码：*service/product.go:257–265*

Q2：ES 挂掉，搜索接口为什么不直接 5xx?

A：搜索是入口，5xx 等于 app 崩页。LIKE 慢一点也比白屏强——业务承诺” 搜索框永远可用”。代码：*service/product.go:294–323*

Q3：hybrid 50/50 这个权重你怎么调?

A：业务值，不是技术值。先 50/50 跑一轮，A/B 观察 CTR / 转化。短 query 偏关键词，长 query 偏语义。代码：*service/search/semantic.go:21–23*

Q4：BM25 和 TF-IDF 在生产里有什么实操差别?

A：BM25 词频走饱和函数，长文档不再天然占便宜；ES 7.x 起默认 BM25。运营说” 排序偏长描述” 先查 `k1/b` 是否被改。

面试 Q&A (2/3)

Q5：min-max 归一在什么场景会失效?

A：所有分数等值时 span=0。代码里返回常量 1，意味着” 两路一样可信”，让另一路决定排序。代码：*service/search/semantic.go:186–192*

Q6：HNSW 的 efSearch 你怎么调?

A：efSearch 是查询时旋钮，不用重建索引。延迟告警时调小（64→32）召回率损 1%–3%，P95 减半。代码：*repository/milvus/product_vector.go:141*

Q7：Milvus 挂了怎么办？为什么不直接 5xx?

A：nopMilvusSearcher 返回空，hybrid 退化为纯 ES。关键词召回不受影响，只丢自然语言长尾。代码：*service/search/milvus_stub.go:19–24*

Q8：换 embedding 模型要走什么流程?

A：维度必须仍是 768（mapping 硬约束），先 staging 跑 NDCG/MRR 评估，再灰度。代码：*service/search/embedding.go:17–24*

面试 Q&A (3/3) + 代码位置

Q9：5 分钟新鲜度 SLA 在哪一步可能掉?

A：堆积一般出现在 RMQ 与 indexer 之间，Qos=32 + ES refresh_interval=1s 下 p95 < 30s。监控 `search.product.indexer` queue_depth。代码：*service/search/indexer.go:25*

Q10：100M SKU 时这个架构还能用吗?

A：ES 切多分片 + 副本；HNSW 内存撑不住改 IVF_PQ 量化；冷热分层把老 SKU 放冷索引。*repository/milvus/product_vector.go:80 NewIndexHNSW*

Q11：以图搜图怎么接?

A：CLIP/SigLIP 编码图片到 512/768 维向量，单独 collection `image_vector`，走相同 outbox 路由。规划中：*repository/milvus/* 新增 *image_vector*

Q12：mapping 改了字段类型怎么办?

A：ES mapping 字段类型不能原地改，必须 `reindex` 或新建索引 + alias 切换。回填走 admin/search/backfill。*routes/router.go:163*

代码位置总览 + 配套材料

搜索 / 语义入口 *routes/router.go:41–42*

ES 检索 + mapping *repository/es/product_index.go:20–41, 163–227*

outbox 事件 *service/product.go:257–265*

indexer 消费 *service/search/indexer.go:17–49*

ES 兜底分支 *service/product.go:294–323*

全量回填 *service/search/backfill.go:13–43*

hybrid 融合 + embedding *service/search/semantic.go, embedding.go*

Milvus HNSW *repository/milvus/product_vector.go:25–28, 80–96, 141–162*

Milvus 接口抽象 *service/search/milvus_stub.go:9–47*

启动静默 *cmd/main.go:64–73 + initialize/search.go:13–31*

配套：deck 04 outbox / saga；deck 10 流量治理与降级；deck 21 AI 检索；stressTest/REPORT.md；docs/blog/10-milvus-vector-search.md；docs/blog/11-semantic-search.md。